

Melody

A programming teamwork in ZJU

游戏简介

Melody 者，基于 Pygame 开发之音游也。

设四轨于屏，音随曲起，符自上而下坠；玩家执键 A / S / K / L，俟符临判线而击之，期与拍合，则中。其旨在使人闻声知节，按之得趣。

项目亮点

- **完整的多场景流程**：主菜单 → 选曲 → 游戏 → 暂停 → 设置/校准，体验闭环清晰。
- **四轨按键玩法**：A / S / K / L 对应四条轨道，节奏紧凑，学习成本低，上手即打。
- **延迟校准**：考虑不同设备音频/输入延迟差异，通过跟拍校准提升判定准确度。
- **暂停回拍机制**：暂停返回时先播放 **8 声节拍器**，帮助重新找回节拍，避免“回来就掉链子”。
- **可维护的模块化组织**：按界面/功能拆分文件，逻辑职责清晰，便于课程作业展示与二次迭代。

操作指南

主菜单

- ↑ / ↓：切换菜单项
- Enter：确认

选曲界面

- ↑ / ↓：选择曲目
- Enter：开始游戏

游戏内

- **A / S / K / L**：四轨道击打
- **Esc**：暂停（进入暂停界面）

暂停界面

- ↑ / ↓：选择功能
- ← / →：调节 **音量 / 本地偏移**
- Enter：继续 / 重开 / 退出
- Esc：继续（返回游戏）

从暂停返回游戏时，会根据**当前节拍**先播放 **8 声节拍器**，再继续游戏。

设置界面

- **Master Volume**: 主音量（左右调整）
- **Latency Calibration**: 延迟校准（按提示跟节拍）

外挂 Auto_play

- Q: 开启/关闭

进入校准后先播放 **4 声节拍器**，随后音符开始下落并可按提示点击校准。

开发过程中遇到的困难

@automataTY

1. 由于此前没有开发软件经历+没有使用pygame库经历+很久没写python了，导致最初立项后问了很多+在b站上学了很久才了解+会用pygame库
2. 在写第一版游玩界面时，由于未引入音符加载和音符绘制/判定分离机制+轨道判定状态机，使得程序在处理绘制和判定音符时逻辑过于复杂，并使得向其中加入新功能极其困难
3. 在开发初期由于思路是一个版块一个版块地写，而非写成函数形式，导致后续在整合不同版块时需要引入main.py中的状态机，造成程序很大部分内容是冗余的
4. 一开始写各个版块时没考虑适应不同窗口大小的问题，导致后续引入修改窗口大小功能时要另辟蹊径才能保证字符间距+字体大小能适应窗口大小

@Silhouette-my

1. 理解 pygame 的函数的参数和返回值比较耗时
2. 原先多次使用 screen，窗口分辨率和字体等参数，造成代码冗余
3. 开发暂停界面时对 ESC 长按和短按的区分，以及遮罩下的可读性、文字排版等有所欠缺
4. 写状态机的时候对返回值没有事先规定好，造成混杂（现在还是需要取别名来避免冲突）

@LotusBreeze

1. 在写进度条函数时，刚开始并没有对total_time进行判断，这导致了total_time等于0时，计算百分比的算式会发生除以0的错误，total_time小于0时，会导致绘制进度条矩形异常，破坏游戏体验。
2. 由于之前没有系统学习Python语言和pygame库函数，因此要先学习pg.font.SysFont和.render以及screen.blit等用于屏幕显示的函数，同时学习了如何实现颜色渐变等功能。

@Aleph-000

1. 窗口尺寸与适配有问题，最大化/改变分辨率后文字消失、布局错位，需要加入 VIDEORESIZE 处理并重新计算布局
2. 代码合并经常出现 merge 冲突标记、return 在函数外、变量未定义等，导致运行中断，后面一处一处校对merge
3. 音量/延迟参数同步时，设置界面与暂停界面音量不一致，延迟校准与游戏判定偏差，需要统一全局状态并正确传递

文件结构

- `main.py`: 主流程与场景切换
- `main_interface.py`: 主界面
- `song_selection.py`: 选曲逻辑
- `setting.py`: 设置与延迟校准
- `pause_interface.py`: 暂停界面
- `play_interface_version2_final_version.py`: 游戏主逻辑

main.py 主流程与场景切换

作为主函数，main.py 以状态机的形式对各个状态的执行逻辑进行了规定

引入资源

开头先进行对库的引用和其它源代码文件的引用：

```
# 引入使用的库
import pygame
import time
import json
import os
import sys

# 引入已有的模块
import main_interface      # 主菜单
import song_selection      # 选曲界面
import setting             # 设置
import play_interface_version2_final_version as play #游戏主界面
import shared_state        # 共享状态
```

状态机定义

然后是对状态机的各个状态进行规定：

```
STATE_MENU = "menu"
STATE_SELECT = "select"
STATE_PLAY = "play"
STATE_RESULT = "result"
STATE_SETTING = "setting"
```

这些状态对应前面引入的模块和功能

初始化画面、状态和参数

这一部分完成了对游戏基础设置（字体、音量、分辨率、延迟）的初始化

```
def main():

    # pygame初始化
    pygame.init()    # pygame自带，对显示、事件、时间、音频、字体等模块进行初始化

    # 定义屏幕和字体
    screen = pygame.display.set_mode((800, 600))
    font = pygame.font.SysFont(None, 50)    # 使用系统字体和英文以增强适配性，避免发生渲染错误（中文出现方框）

    # 初始化状态和参数
    state = STATE_MENU    # 设置主菜单状态
    title_flag = 0    #
    selected_song = None    # 初始化选歌
    master_volume = 1.0    # 全局音量
    current_latency = 0    # 初始化延迟值（个体差异）
    local_offset = 0    # 初始化本地延迟（机器差异）
    screen_size = (800, 600)    # 设置窗口分辨率

    if pygame.mixer.get_init():    # 如果音频模块已初始化
        pygame.mixer.music.set_volume(master_volume)    # 设置全局音量为前面初始化的结果
    shared_state.MASTER_VOLUME = master_volume    # 同步
```

主循环

对各个状态机的执行进行了定义

主菜单

初始化

初始化屏幕、界面选择相关以及 ESC 的判定

```
while True:
    if state == STATE_MENU:
        # 调用主菜单界面
        screen.fill((0, 0, 0)) # 填充屏幕为全黑
        text_rect = main_interface.screen_interface(screen, font)
        # 调用主界面中的函数绘制UI
        last_rect = main_interface.button_border_draw(screen, text_rect, 0)
        # 绘制按钮的高亮边框
        selected_index = 0 # 初始化“选中的按钮”索引为第一个按钮

        # 初始化esc操作
        esc_hold_start = None
        esc_short_action = None
```

主状态

根据不同按钮定义不同行为

关闭按钮：退出

Enter：确认

上下键：选择

ESC：短按返回，长按退出

```
running = True
while running: # 游戏进行中
    for ev in pygame.event.get(): # 遍历所有事件

        if ev.type == pygame.QUIT: # 用户点击关闭按钮
            # pygame常用的退出
            pygame.quit() # 释放pygame模块
            sys.exit() # 退出程序

        elif ev.type == pygame.KEYDOWN: # 用户按下键盘

            if ev.key == pygame.K_RETURN: # Enter 进入选曲
                if selected_index == 0: # 进入选曲界面
                    state = STATE_SELECT
                    running = False
                elif selected_index == 1: # 进入设置界面
                    state = STATE_SETTING
                    running = False
                elif selected_index == 2: # 返回上一层
                    return

            elif ev.key == pygame.K_ESCAPE: # ESC键
```

```

        esc_hold_start = pygame.time.get_ticks()    # 记录按下
时间区分长按短按

        esc_short_action = "quit"    #    如果短按就退出

    elif ev.key == pygame.K_DOWN or ev.key == pygame.K_UP:    #
上下键移动菜单

        if ev.key == pygame.K_DOWN and selected_index <
len(text_rect) - 1:    # 下移（不在边界）
            main_interface.button_border_clear(screen,
last_rect)    # 清除旧高亮边框

            selected_index += 1
            last_rect =
main_interface.button_border_draw(screen, text_rect, selected_index)    # 重新绘制
高亮边框

        elif ev.key == pygame.K_UP and selected_index > 0:    #
上移（不到边界）

            main_interface.button_border_clear(screen,
last_rect)

            selected_index -= 1
            last_rect =
main_interface.button_border_draw(screen, text_rect, selected_index)

    elif ev.type == pygame.KEYUP and ev.key == pygame.K_ESCAPE:    #
松开ESC

        if esc_hold_start is not None and pygame.time.get_ticks()
- esc_hold_start < 2000:    # 短按小于两秒，返回上一级
            return

        # 长按则清空状态
        esc_hold_start = None
        esc_short_action = None

    # 持续检测ESC按键状态
    keys = pygame.key.get_pressed()
    if keys[pygame.K_ESCAPE] and esc_hold_start is not None:
        if pygame.time.get_ticks() - esc_hold_start >= 2000:
            # 长按大于两秒，退出
            pygame.quit()
            sys.exit()
    elif not keys[pygame.K_ESCAPE]:
        esc_hold_start = None    # 松开ESC就退出
pygame.display.update()    # 更新屏幕

```

选曲界面

初始化

```
elif state == STATE_SELECT:
    # 调用选曲界面
    clock = pygame.time.Clock() # 创建控制帧率的时钟对象
    running = True
    # 初始化ESC检测
    esc_hold_start = None
    esc_short_action = None
```

主状态

和主菜单一样的逻辑

```
while running:
    for ev in pygame.event.get():
        if ev.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        elif ev.type == pygame.KEYDOWN:
            if ev.key == pygame.K_RETURN:
                # 确认选曲
                selected_song = song_selection.file_play[title_flag]
                # 导入选歌模块中的函数选歌
                state = STATE_PLAY # 跳转到游玩模块
                running = False

            elif ev.key == pygame.K_ESCAPE: # ESC 短按返回
                esc_hold_start = pygame.time.get_ticks()
                esc_short_action = "back"

            # 上下切换
            elif ev.key == pygame.K_DOWN:
                title_flag = min(title_flag+1,
len(song_selection.file_play)-1)

            elif ev.key == pygame.K_UP:
                title_flag = max(title_flag-1, 0)

            # ESC判定
            elif ev.type == pygame.KEYUP and ev.key == pygame.K_ESCAPE:
                if esc_hold_start is not None and pygame.time.get_ticks()
- esc_hold_start < 2000:
                    state = STATE_MENU
                    running = False
                    esc_hold_start = None
                    esc_short_action = None
```

```

        # 界面渲染
        screen.fill((0,0,0))
        song_selection.text_draw(title_flag,
pygame.font.SysFont(None,50), screen_size, 0) # 绘制列表文字
        song_selection.attention_draw(screen,screen_size,0) #高亮选中歌曲

    # 检测ESC
    keys = pygame.key.get_pressed()
    if keys[pygame.K_ESCAPE] and esc_hold_start is not None:
        if pygame.time.get_ticks() - esc_hold_start >= 2000:
            pygame.quit()
            sys.exit()
    elif not keys[pygame.K_ESCAPE]:
        esc_hold_start = None

    # 刷新，帧率设为60
    pygame.display.update()
    clock.tick(60)

```

设置界面

通过维护字典 settings 存储用户的设置，更新并将其传入，并根据设置调整参数。

```

elif state == STATE_SETTING:
    # 调用设置模块，返回字典 settings
    settings = setting.run_settings(master_volume, current_latency,
screen_size)

    if isinstance(settings, dict): # 检查settings是不是字典
        # 传入更新的变量
        if "master_volume" in settings:
            master_volume = settings["master_volume"]
            shared_state.MASTER_VOLUME = master_volume
        if "latency_ms" in settings:
            current_latency = settings["latency_ms"]
        if "screen_size" in settings:
            screen_size = settings["screen_size"]

    # 用新的屏幕分辨率重新初始化
    pygame.init()
    screen = pygame.display.set_mode(screen_size)
    font = pygame.font.SysFont(None, 50)

    # 更新新的音量
    if pygame.mixer.get_init():
        pygame.mixer.music.set_volume(master_volume)

    state = STATE_MENU #返回主菜单

```


游玩界面

读取 json

从目录下保存的歌曲 json（谱面）中获取相关信息

```
elif state == STATE_PLAY:

    master_volume = shared_state.MASTER_VOLUME # 从共享状态中获取音量值

    # 读取选中的歌曲json
    with open(selected_song, 'r') as f: # 以只读模式打开，文件对象赋值给f
        get_content = json.load(f) # 从json中读取数据并解析成python对象
        note_file = get_content['note'] # 读取note
        length_temp = len(note_file) # 读取音符数（用来找到最后一个）
        local_offset = note_file[length_temp-1]['offset'] #根据最后一个音符确定本地偏移量
```

调用游玩模块，更新音量

```
result, new_local_offset = play.run_game(selected_song, master_volume,
current_latency, local_offset, screen_size) # 调用游玩模块

# 更新音量
if pygame.mixer.get_init():
    master_volume = pygame.mixer.music.get_volume()
    shared_state.MASTER_VOLUME = master_volume
```

处理重新开始部分

重新加载 json，更新本地偏移

```
# 重新开始
if result == "restart":
    with open(selected_song, 'r') as f:
        data = json.load(f)
        length_temp = len(data['note'])
        data['note'][length_temp-1]['offset'] = new_local_offset #
更新此时最后一个音符的偏移量（因为打到一半偏移量会变）
    with open(selected_song, 'w') as f:
        json.dump(data, f) # 将其写回文件
    local_offset = new_local_offset # 更新本地偏移
    continue # 回到循环开头，即继续运行
```

结束后收拾

```
# 游戏结束后重置环境
pygame.init()
screen = pygame.display.set_mode(screen_size)
font = pygame.font.SysFont(None, 50)
if pygame.mixer.get_init():
    pygame.mixer.music.set_volume(master_volume)
state = STATE_SELECT    # 回到select
```

结果界面（并未启用，留作更新内容）

初始化

```
# STATE_RESULT可删/留着做标准结果呈现界面
elif state == STATE_RESULT:
    # 简单结果界面
    font = pygame.font.SysFont(None, 50)
    text = font.render("Game Over - Press Enter to return", True,
'white')

    rect = text.get_rect(center=screen.get_rect().center)
    screen.blit(text, rect)
    pygame.display.update()
    waiting = True    # 等待
    esc_hold_start = None
    esc_short_action = None
```

反应

退出按钮：退出

Enter：回到主菜单

ESC：短按返回，长按退出

```
while waiting:
    for ev in pygame.event.get():
        if ev.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        elif ev.type == pygame.KEYDOWN and ev.key == pygame.K_RETURN:
            state = STATE_MENU
            waiting = False
        elif ev.type == pygame.KEYDOWN and ev.key == pygame.K_ESCAPE:
            esc_hold_start = pygame.time.get_ticks()
            esc_short_action = "back"
        elif ev.type == pygame.KEYUP and ev.key == pygame.K_ESCAPE:
            if esc_hold_start is not None and pygame.time.get_ticks()
- esc_hold_start < 2000:
                state = STATE_MENU
                waiting = False
                esc_hold_start = None
```

```

        esc_short_action = None
    keys = pygame.key.get_pressed()
    if keys[pygame.K_ESCAPE] and esc_hold_start is not None:
        if pygame.time.get_ticks() - esc_hold_start >= 2000:
            pygame.quit()
            sys.exit()
    elif not keys[pygame.K_ESCAPE]:
        esc_hold_start = None

```

结尾

```

if __name__ == "__main__":
    main()

```

防止被别的函数调用

pause_interface.py 暂停界面

引入库

```

import json as js
import os
import numpy as np
import pygame as pg
import time
import shared_state
import sys

```

text_draw 逐条渲染文本

传入参数

```
def text_draw(screen, font, text_use, coordinate, rect_text_use):
```

- screen: 屏幕上对象
- font: 渲染文本使用的字体
- text_use: 需要绘制的文本内容列表
- coordinate: 坐标与对齐方式列表
- rect_text_use: 外部传入, 文本的Rect(位置和大小)

函数体

根据参数渲染文本、确定位置并画到screen上

```

# 获取屏幕的宽高
width = pg.Surface.get_width(screen)
height = pg.Surface.get_height(screen)

```

```

# 遍历文本进行渲染
for i in range(len(text_use)):
    text = text_use[i]
    text_image = font.render(text, True, 'white') # True意为使用抗锯齿
    text_rect = text_image.get_rect()

    # 注意：这里x坐标应该是绝对坐标，不是比例
    x_pos = coordinate[i][0]
    y_pos = coordinate[i][1]

    if coordinate[i][2] == 'right': # 右对齐
        text_rect.right = x_pos
        text_rect.centery = y_pos
    elif coordinate[i][2] == 'center': # 居中
        text_rect.center = (x_pos, y_pos)

    rect_text_use.append(text_rect) # 将每个文本的最终位置矩形保存到rect_text_use

# 绘制到surface（即被创建的 screen）
surface = pg.display.get_surface()
if surface is not None:
    surface.blit(text_image, text_rect) #把text_image 画到 text_rect 标记的区域

```

coordinate_calculate 计算位置和对齐方式

传入参数

```
def coordinate_calculate(width,height,font,coordinate_text_use):
```

- width: 屏幕宽度
- height: 屏幕高度
- font: 使用字体
- coordinate_text_use: 存储得到的坐标格式

函数体

根据得到的宽和高计算按钮的位置，将其存入coordinate_text_use

```

# 计算中间矩形区域（9宫格的正中间）
# 中间矩形左上角坐标: (width/3, height/3)
# 中间矩形尺寸: (width/3, height/3)
center_rect_x = width / 3
center_rect_y = height / 3
center_rect_width = width / 3
center_rect_height = height / 3

# 计算文字坐标，格式: [x位置, y位置, 对齐方式]

```

```

# 1. volume - 在中间矩形左侧，右对齐，y在矩形上1/4处
# 右对齐的x坐标应该是中间矩形的左边界
vol_x = center_rect_x
vol_y = center_rect_y + center_rect_height * 1/3
coordinate_text_use.append([vol_x, vol_y, 'right'])

# 2. local offset - 在中间矩形左侧，右对齐，y在矩形上1/2处
local_x = center_rect_x
local_y = center_rect_y + center_rect_height * 2/3
coordinate_text_use.append([local_x, local_y, 'right'])

# 3. continue, restart, quit - 在中间矩形底部，均分，居中
button_bottom_y = center_rect_y + center_rect_height # 底部y坐标

# 首先获取每个按钮文本的宽度，用于计算总宽度
button_texts = ['continue', 'restart', 'quit']
button_widths = []
for text in button_texts:
    text_image = font.render(text, True, 'white')
    button_widths.append(text_image.get_width())

# 计算三个按钮的总宽度和平均间隔
total_buttons_width = sum(button_widths)

# 在中间矩形内等间距排列三个按钮
# 左侧起始位置：中间矩形左边界
left_boundary = center_rect_x
# 右侧结束位置：中间矩形右边界
right_boundary = center_rect_x + center_rect_width
# 可用总宽度：中间矩形宽度
available_width = center_rect_width

# 让每个按钮之间的间距相等（按钮间有4个间隔：左边界-按钮1，按钮1-按钮2，按钮2-按钮3，按钮
3-右边界）
total_gap = available_width - total_buttons_width
gap_width = total_gap / 4 # 4个间隔

# 计算每个按钮的x位置
current_x = left_boundary + gap_width
button_positions = []

for i in range(3):
    # 按钮的x中心位置 = current_x + 按钮宽度的一半
    button_center_x = current_x + button_widths[i] / 2
    button_positions.append(button_center_x)

    # 移动到下一个按钮的起始位置
    current_x += button_widths[i] + gap_width

# 添加三个按钮的坐标
for i in range(3):

```

```
coordinate_text_use.append([button_positions[i], button_bottom_y,
'center'])
```

button_border_draw 绘制高亮框并返回

传入参数

```
def button_border_draw(screen, text_rect, select_flag):
```

- screen
- text_rect: select_draw 生成的存放所有文本矩形的列表
- select_flag: 当前选中文本的索引

函数体

```
# 扩展边界，这样边框不紧贴文字，更加美观
border_x = text_rect[select_flag].x-5
border_y = text_rect[select_flag].y-5
border_width = text_rect[select_flag].width+10
border_height = text_rect[select_flag].height+10

# 创建边框矩形对象
last_rect = pg.Rect(border_x, border_y, border_width, border_height)

# 绘制边框
border_color = 'white'
border_line_width = 1 # 指定宽度，避免变成填充
pg.draw.rect(screen, border_color, last_rect, border_line_width)

return last_rect # 返回边框矩形
```

button_border_clear 清除高亮框（改成黑色）

```
def button_border_clear(screen, last_rect):
    pg.draw.rect(screen, 'black', last_rect, 1)
```

slider_draw 绘制滑动条

传入参数

```
def
slider_draw(screen, value, label, rect_text_use, font, slider_volume_rect, slider_offset_r
ect):
```

- screen
- value: 滑动条当前数值
- label: 滑动条标签 (volume or offset)

- rect_text_use
- font
- slider_volume_rect: 存放音量条相关矩形
- slider_offset_rect: 存放偏移调相关矩形

函数体

```
# 获取宽高
width = pg.Surface.get_width(screen)
height = pg.Surface.get_height(screen)

#绘制音量条
if(label == 'volume'):
    # 获取continue和quit按钮的矩形, rect_text_use[2]是continue, rect_text_use[4]
是quit
    continue_rect = rect_text_use[2]
    quit_rect = rect_text_use[4]

    # 滑动条的左端与continue左端对齐, 右端与quit右端对齐
    slider_left = continue_rect.left
    slider_right = quit_rect.right

    # 计算宽度(百分比)
    slider_width = (slider_right - slider_left)*value/100

    # 滑动条的高度与音量文本的高度相同, y位置与音量文本对齐
    slider_height = rect_text_use[0].height
    slider_y = rect_text_use[0].y

    # 创建滑动条矩形
    slider_rect = pg.Rect(slider_left, slider_y, slider_width, slider_height)

    # 渲染创建数值矩形
    volume_image = font.render(str(value), True, 'white')
    volume_rect = volume_image.get_rect()
    volume_rect.x = slider_right+20    # 在滑动条右侧显示数值
    volume_rect.y = slider_rect.y

    # 把滑动条和数值矩形保存到列表
    slider_volume_rect.append(slider_rect)
    slider_volume_rect.append(volume_rect)

    # 绘制滑动条和数值矩形
    pg.draw.rect(screen, 'white', slider_rect)
    surface = pg.display.get_surface()
    if surface is not None:
        surface.blit(volume_image, volume_rect)

# 绘制偏移条(几乎同上)
```

```

elif(label == 'local offset'):
    continue_rect = rect_text_use[2]
    quit_rect = rect_text_use[4]

    slider_left = continue_rect.left
    slider_right = quit_rect.right
    slider_width = slider_right - slider_left

    slider_height = rect_text_use[1].height
    slider_y = rect_text_use[1].y

    # 创建滑动条矩形
    slider_rect = pg.Rect(slider_left,slider_y,slider_width,slider_height)

    # 分割左右部分
    slider_left_rect = pg.Rect(slider_left, slider_y, slider_width*
(0.5+value/2000), slider_height)
    slider_right_rect = pg.Rect(slider_left+slider_width*
(0.5+value/2000),slider_y,slider_width*(0.5-value/2000),slider_height)

    # 渲染创建数值矩形
    offset_image = font.render(str(value), True, 'white')
    offset_rect = offset_image.get_rect()
    offset_rect.x = slider_right+20
    offset_rect.y = slider_rect.y

    slider_offset_rect.append(slider_rect)
    slider_offset_rect.append(offset_rect)

    # 绘制偏移条和数值矩形
    pg.draw.rect(screen, 'white', slider_left_rect)
    pg.draw.rect(screen, 'blue', slider_right_rect)
    surface = pg.display.get_surface()
    if surface is not None:
        surface.blit(offset_image, offset_rect)

```


slider_clear 清除滑动条

```
def slider_clear(screen,label,slider_volume_rect,slider_offset_rect):
    if(label == 'volume'):
        for i in range(0,2,1):
            pg.draw.rect(screen,'black',slider_volume_rect[i]) # 涂黑
        for i in range(0,len(slider_volume_rect),1):
            del slider_volume_rect[0] # 清空矩形列表

    # 下面同上
    elif(label == 'local offset'):
        for i in range(0,2,1):
            pg.draw.rect(screen,'black',slider_offset_rect[i])
        for i in range(0,len(slider_offset_rect),1):
            del slider_offset_rect[0]
```

run_pause 暂停核心函数

传入参数

```
def run_pause(screen, current_volume, current_offset):
```

- screen
- current_volume: 字面意思
- current_offset: 同上

初始化

```
# 获取宽高
width = pg.Surface.get_width(screen)
height = pg.Surface.get_height(screen)

clock = pg.time.Clock() # 创建时钟对象控制帧率

# 针对不同分辨率选择不同字号
font_size = [30,40,60]
screen_sizes = [(800,600),(1280,760),(1920,1080)]
font_size_use = None
for i in range(0,3,1):
    size = screen_sizes[i]
    if(size == pg.Surface.get_size(screen)):
        font_size_use = font_size[i]
        break
font = pg.font.SysFont(None,font_size_use)
screen.fill((0, 0, 0)) # 清屏

# 文本与坐标初始化
text_use = ['volume','local offset','continue','restart','quit'] # 菜
单中文本
```

```

coordinate_text_use = list()      # 坐标列表
rect_text_use = list()           # 矩形列表
slider_volume_rect = list()      # 滑动条矩形列表
slider_offset_rect = list()

# 初始化音量和偏移值
volume = max(0, min(100, int(round(shared_state.MASTER_VOLUME * 100))))
offset = int(current_offset)

flag = 0      # 选中的菜单项索引

coordinate_calculate(width,height,font,coordinate_text_use)      # 计算文本坐标
text_draw(screen,font,text_use,coordinate_text_use,rect_text_use)      # 绘制文本

# 绘制滑动条

slider_draw(screen,volume,'volume',rect_text_use,font,slider_volume_rect,slider_offset_rect)
slider_draw(screen,offset,'local offset',rect_text_use,font,slider_volume_rect,slider_offset_rect)

last_rect = button_border_draw(screen,rect_text_use,flag)      # 绘制高亮框

# 参数初始化
last_update_time = 0      # 上次按键更新时间
key_update_delay = 80      # 按键延迟（避免过快重复）
action = "resume"      # 返回的操作（默认继续）
esc_hold_start = None      # ESC按下时间

```

主循环

```

isRunning = True
while isRunning:
    current_time = pg.time.get_ticks()

    # 处理键盘事件(同主菜单)
    for ev in pg.event.get():
        if(ev.type == pg.QUIT):
            pg.quit()
            sys.exit()
        elif(ev.type == pg.KEYDOWN):
            if(ev.key == pg.K_ESCAPE):
                esc_hold_start = pg.time.get_ticks()
            elif(ev.key == pg.K_DOWN):
                button_border_clear(screen,last_rect)
                flag = min(4,flag+1)
                last_rect = button_border_draw(screen,rect_text_use,flag)
            elif(ev.key == pg.K_UP):
                button_border_clear(screen,last_rect)

```

```

        flag = max(0, flag-1)
        last_rect = button_border_draw(screen, rect_text_use, flag)
    elif(ev.key == pg.K_RETURN):
        if(flag == 2):
            action = "resume"
        elif(flag == 3):
            action = "restart"
        elif(flag == 4):
            action = "quit"
        isRunning = False
        break
    elif(ev.type == pg.KEYUP and ev.key == pg.K_ESCAPE):
        if esc_hold_start is not None and pg.time.get_ticks() -
esc_hold_start < 2000:
            action = "resume"
            isRunning = False
            break
        esc_hold_start = None

# 按键检测
if(current_time - last_update_time > key_update_delay):
    keys = pg.key.get_pressed()
    if keys[pg.K_RIGHT]:      # 右键（调节）
        if(flag == 0):      # 调节音量

slider_clear(screen, 'volume', slider_volume_rect, slider_offset_rect)
        volume = min(100, volume+1)

slider_draw(screen, volume, 'volume', rect_text_use, font, slider_volume_rect, slider_o
ffset_rect)

        if pg.mixer.get_init():
            pg.mixer.music.set_volume(volume/100)
            shared_state.MASTER_VOLUME = volume / 100.0      # 同步音量

        elif(flag == 1):      # 调节偏移
            slider_clear(screen, 'local
offset', slider_volume_rect, slider_offset_rect)
            offset = min(1000, offset+10)
            slider_draw(screen, offset, 'local
offset', rect_text_use, font, slider_volume_rect, slider_offset_rect)

        elif keys[pg.K_LEFT]:      # 左键同理
            if(flag == 0):

slider_clear(screen, 'volume', slider_volume_rect, slider_offset_rect)
            volume = max(0, volume-1)

slider_draw(screen, volume, 'volume', rect_text_use, font, slider_volume_rect, slider_o
ffset_rect)

        if pg.mixer.get_init():
            pg.mixer.music.set_volume(volume/100)

```

```

        shared_state.MASTER_VOLUME = volume / 100.0
    elif(flag == 1):
        slider_clear(screen,'local
offset',slider_volume_rect,slider_offset_rect)
        offset = max(-1000,offset-10)
        slider_draw(screen,offset,'local
offset',rect_text_use,font,slider_volume_rect,slider_offset_rect)

        last_update_time = current_time    #更新修改时间

    # 检测ESC
    keys = pg.key.get_pressed()
    if keys[pg.K_ESCAPE] and esc_hold_start is not None:
        if pg.time.get_ticks() - esc_hold_start >= 2000:
            pg.quit()
            sys.exit()
    elif not keys[pg.K_ESCAPE]:
        esc_hold_start = None

    pg.display.update()    # 刷新帧
    clock.tick(60)        # 限制帧率

    shared_state.MASTER_VOLUME = volume / 100.0
    return action, volume/100.0, offset    # 返回操作与设置

```

主函数

定义与初始化

```

pg.init()

# 定义菜单内容和参数
text_use = ['volume','local offset','continue','restart','quit']
coordinate_text_use = list()
rect_text_use = list()
slider_volume_rect = list()
slider_offset_rect = list()
volume = 50
offset = 0

```

主程序

```

if __name__ == "__main__":    # 直接运行时才会执行
    screen_size = [(800,600),(1280,760),(1920,1080)]
    size_select = 1    # 默认分辨率
    screen = pg.display.set_mode(screen_size[size_select])    # 创建screen
    run_pause(screen, 0.5, 0)    # 调用暂停
    pg.quit()

```

setting.py 设置与延迟校准

`setting.py` 模块负责处理游戏的设置选项（如音量、分辨率）以及核心的音频延迟校准功能。它通过返回一个包含设置信息的字典与主程序进行交互。

引入库

```
import pygame as pg
import shared_state
import sys
```

- `pygame`：用于图形渲染、事件处理和音频播放。
- `shared_state`：引用全局共享变量（如 `MASTER_VOLUME`），保证音量设置能在全局生效。
- `sys`：用于在必要时退出程序。

辅助绘图函数

`_draw_text_centered` 居中绘制文本

封装了基础的文本渲染逻辑，减少重复代码。

```
def _draw_text_centered(screen, font, text, center):
    text_image = font.render(text, True, "white") # 渲染白色文本
    text_rect = text_image.get_rect(center=center) # 获取居中矩形
    screen.blit(text_image, text_rect) # 绘制到屏幕
    return text_rect # 返回矩形区域供后续使用（如计算边框）
```

`_button_border_draw` 绘制按钮边框

用于在当前选中的菜单项周围绘制一个白色边框，提示玩家当前的交互焦点。

```
def _button_border_draw(screen, text_rect, select_flag):
    # 计算边框位置，向外扩展 5 像素以留出留白
    border_x = text_rect[select_flag].x - 5
    border_y = text_rect[select_flag].y - 5
    border_width = text_rect[select_flag].width + 10
    border_height = text_rect[select_flag].height + 10

    last_rect = pg.Rect(border_x, border_y, border_width, border_height)
    pg.draw.rect(screen, "white", last_rect, 1) # 绘制线宽为1的空心矩形
    return last_rect
```

`_draw_volume_row` 绘制音量调节行

这是一个专门用于绘制“音量调节”这一行的复杂绘制函数。它不仅显示文字，还动态绘制一个代表音量的进度条。

```
def _draw_volume_row(screen, font, label, volume, center):
    # 将 0.0-1.0 的音量转换为 0-100 的整数
    value = max(0, min(100, int(round(volume * 100))))

    # 渲染标签和数值文字
    label_image = font.render(label, True, "white")
    value_image = font.render(str(value), True, "white")

    # 动态计算布局，确保整体居中
    # ... (计算 slider_width, start_x 等坐标逻辑) ...

    # 绘制标签
    screen.blit(label_image, label_rect)
    # 绘制代表音量的白色实心矩形（进度条）
    pg.draw.rect(screen, "white", slider_rect)
    # 绘制数值
    screen.blit(value_image, value_rect)

    # 返回包含整行的矩形区域，用于 _button_border_draw 判定高亮范围
    return pg.Rect(start_x, label_rect.y, total_width, label_rect.height)
```

界面渲染

`_render_menu` 渲染设置菜单主界面

该函数每帧被调用，负责将所有菜单项画到屏幕上。

```
def _render_menu(screen, font, small_font, volume, latency_ms, size_label,
selected_index):
    screen.fill((0, 0, 0)) # 清屏

    # 绘制标题
    width, height = screen.get_size()
    _draw_text_centered(screen, font, "Settings", (width // 2, height // 6))

    # 定义菜单项内容
    items = [
        f"Latency Calibration: {latency_ms} ms", # 显示当前校准的延迟
        f"Screen Size: {size_label}", # 显示当前分辨率
        "Back",
    ]

    rects = []
    base_y = height // 2
```

```

# 1. 绘制音量行（第一项，索引0）
rects.append(_draw_volume_row(screen, font, "Master Volume", volume, (width
// 2, base_y)))

# 2. 绘制其余文本选项
for i, text in enumerate(items, start=1):
    rects.append(_draw_text_centered(screen, font, text, (width // 2, base_y
+ i * 60)))

# 绘制底部操作提示
_draw_text_centered(screen, small_font, "Use Up/Down to select, Left/Right to
adjust volume, Enter to confirm", (width // 2, height - 40))

# 绘制当前选中项的边框
_button_border_draw(screen, rects, selected_index)

return rects

```

延迟校准逻辑

`_run_latency_calibration` 延迟校准微型游戏

这是本模块的核心算法部分。它运行一个独立的循环，播放节拍并让玩家按下空格键，计算玩家输入与实际节拍的时间差，从而得出设备的音频延迟。

1. 音频合成

为了不依赖外部文件，代码直接生成方波音频作为节拍音效：

```

# 生成 880Hz 的方波数据
sample = 16000 if (i * freq * 2 // sample_rate) % 2 == 0 else -16000
buf[i * 2:i * 2 + 2] = int(sample).to_bytes(2, byteorder="little",
signed=True)
beep = pg.mixer.Sound(buffer=bytes(buf))

```

2. 视觉与时间同步

- `pre_beats`：预备拍（4拍），只响不判定。
- `num_beats`：测试拍（6拍），用于记录玩家点击。
- `speed`：计算圆圈下落速度，使其准确在节拍时间点到达判定线 `target_y`。

3. 判定循环

```

while True:
    now = pg.time.get_ticks()

    # 播放节拍音效逻辑
    # ... 检查时间是否到达 pre_beat_times 或 beat_times ...

    # 输入检测

```

```

for ev in pg.event.get():
    if ev.type == pg.KEYDOWN:
        # 玩家在听到节拍时按下 Space 或 Enter
        if ev.key in (pg.K_SPACE, pg.K_RETURN) and beat_index <
num_beats:
            offset = now - beat_times[beat_index] # 计算 实际按下时间 - 理论
节拍时间

            hits.append(offset)
            beat_index += 1

# 视觉绘制：绘制移动的圆圈和判定线
pg.draw.line(screen, "white", (width // 2 - 60, target_y), (width // 2 +
60, target_y), 2) # 判定线
for beat_time in beat_times:
    # 根据时间差计算圆圈的 Y 坐标
    y = target_y - dt * speed
    pg.draw.circle(screen, "white", (width // 2, int(y)), 12, 2)

# 结果计算
if beat_index >= num_beats and not waiting_for_result:
    # 计算平均偏差作为延迟值
    avg = sum(hits) / float(len(hits)) if hits else 0.0
    current_latency = int(round(avg))
    waiting_for_result = True # 进入结果展示状态

```

主设置循环

run_settings 设置模块入口

管理设置界面的主循环、事件监听和状态更新。

参数：

- `master_volume`, `latency_ms`, `screen_size`：当前的设置状态。

逻辑：

1. **初始化**：设置屏幕模式和字体。
2. **事件循环**：
 - `↑ / ↓`：修改 `selected_index` 切换菜单项。
 - `← / →`：
 - 若选中 **Master Volume**：直接修改 `master_volume` 变量，并调用 `pg.mixer.music.set_volume` 实时反馈，同时更新 `shared_state`。
 - 若选中 **Screen Size**：切换分辨率索引，并立即调用 `pg.display.set_mode` 应用新分辨率。
 - **Enter**：
 - 若选中 **Latency Calibration**：调用 `_run_latency_calibration`，并将返回的新延迟值存入 `latency_ms`。

- 若选中 **Back**: 退出循环。
- **Esc**: 长按 2 秒退出或短按返回。

3. 返回结果:

函数最终返回一个字典, 包含修改后的所有设置, 供 `main.py` 更新全局状态:

```
return {  
    "master_volume": master_volume,  
    "latency_ms": latency_ms,  
    "screen_size": screen_sizes[size_select],  
}
```

main_interface.py 主界面

`main_interface.py` 负责绘制游戏的主菜单界面, 包括游戏标题 "Melody" 和三个核心选项 (Start, Settings, Exit)。它实现了自适应屏幕分辨率的布局逻辑。

引入库

```
import json as js  
import os  
import numpy as np  
import pygame as pg
```

- `pygame`: 核心图形库。
- `numpy`: 虽然引入了但在此模块中未深度使用 (可能是遗留代码)。

全局配置

```
MENU_ITEMS = ["Start", "Settings", "Exit"]
```

定义了主菜单显示的三个选项文本。

辅助函数

`_get_scale_factor` 获取缩放因子

为了适配不同分辨率 (如 800x600, 1280x760, 1920x1080), 该函数计算当前屏幕尺寸相对于基准尺寸 (800x600) 的缩放比例。

```
def _get_scale_factor(screen_size):  
    base_width, base_height = 800, 600  
    width, height = screen_size  
    # 取宽和高缩放比例的较小值, 保证画面不被拉伸变形  
    scale_w = width / base_width  
    scale_h = height / base_height  
    return min(scale_w, scale_h)
```

界面绘制逻辑

screen_interface 绘制主界面

这是绘制静态UI的核心函数。它负责渲染标题和菜单按钮，并根据屏幕宽度自动计算间距。

参数：

- `screen`：绘制的目标表面。
- `font`：传入的基础字体对象（虽然函数内部重新计算了大小）。

主要逻辑：

1. 动态计算字号：

根据 `_get_scale_factor` 计算出的比例，动态调整菜单字体（基准50）和标题字体（基准80）的大小。

```
scale_factor = _get_scale_factor(pg.Surface.get_size(screen))
font_size = int(50 * scale_factor)
font_title = pg.font.SysFont(None, int(80*scale_factor))
```

2. 绘制标题：

将 "Melody" 绘制在屏幕水平居中、垂直中心偏上的位置。

```
title = "Melody"
title_rect.center = (mid_pos[0], mid_pos[1] - y_offset)
screen.blit(title_image, title_rect)
```

3. 计算菜单布局：

为了使菜单项水平排列且居中，先计算所有文本的总宽度和间隔。

- `item_widths`：收集每个单词的宽度。
- `spacing`：根据屏幕宽度动态计算间隔 (`s_width / 15 * scale_factor`)。
- `start_x`：计算整体的起始 X 坐标，公式为 `中点 - (总字宽 + 总间距)/2`。

4. 绘制菜单项：

遍历 `MENU_ITEMS`，依次在计算好的位置绘制文本，并将每个文本的 `Rect` 对象存储在 `text_rect` 列表中返回。

```
for i, item in enumerate(text):
    # ... 渲染文本 ...
    text_rects.center = (current_x + item_widths[i]/2, mid_pos[1] + y_offset)
    screen.blit(text_images, text_rects)
    # 更新下一个 X 坐标
    current_x += item_widths[i] + spacing
```

交互反馈

button_border_draw 绘制选中框

在被选中的菜单项周围绘制一个白色边框。

```
def button_border_draw(screen, text_rect, select_flag):
    # 根据缩放因子调整边框的内边距(padding)和线宽
    scale_factor = _get_scale_factor(pg.Surface.get_size(screen))
    border_x = text_rect[select_flag].x - int(5 * scale_factor)
    # ... (计算 border_y, border_width, border_height)

    last_rect = pg.Rect(...)
    pg.draw.rect(screen, border_color, last_rect, border_line_width)
    return last_rect # 返回边框区域用于清除
```

button_border_clear 清除选中框

用黑色矩形覆盖上一次绘制的边框，用于在切换选项时清除旧的高亮。

```
def button_border_clear(screen, last_rect):
    pg.draw.rect(screen, 'black', last_rect, 1) # 注意：这里用黑色重绘边框
```

独立测试模块

`if __name__ == "__main__":` 块包含了一个独立的测试循环，允许直接运行此文件来预览界面效果。

- **初始化：**设置 800x600 窗口。
- **事件循环：**
 - `VIDEORESIZE`：监听窗口大小改变，重新调用 `screen_interface` 重绘界面，实现响应式布局。
 - `KEYDOWN` (Right/Left)：模拟主程序中的菜单切换逻辑，测试 `button_border_draw` 和 `button_border_clear` 的效果。

song_selection.py 选曲界面

`song_selection.py` 模块负责扫描游戏目录下的谱面文件，并提供一个可视化的列表供玩家选择曲目。它实现了歌曲标题的解析、列表导航以及简单的视觉特效（如文字闪烁）。

引入库与全局初始化

```
import json as js
import os
import numpy as py
import pygame as pg
```

在模块加载时，脚本会立即执行文件扫描逻辑，以便其他模块（如 `main.py`）导入时能直接获取歌曲列表。

1. 扫描谱面文件

```
# 获取当前工作目录
root = os.getcwd()
path = os.listdir(root)

# 初始化存储列表
file_play = [] # 存储谱面文件路径 (e.g., "song1.json")
title_song = [] # 存储解析出的曲目标题 (e.g., "My Song")

for p in path:
    type_file = os.path.splitext(p)
    if type_file[1] == '.json': # 筛选 .json 文件
        try:
            with open(p, 'r', encoding='utf-8') as file:
                get_content = js.load(file)
            # 提取 meta.song.title 字段
            title = get_content.get('meta', {}).get('song', {}).get('title')
            if title:
                file_play.append(p)
                title_song.append(title)
        # ... 异常处理 ...
```

这段代码遍历文件夹，打开每一个 `.json` 文件，解析其元数据（Meta Data），将有效的文件名和对应的歌曲标题分别存入 `file_play` 和 `title_song` 列表供外部调用。

辅助逻辑函数

flag_judge 边界判断

用于检测当前选中的索引（`flag_now`）是否到达了列表的顶部或底部，防止数组越界。

```
def flag_judge(flag_now, file_play_len, lower):
    # 如果到达下界(lower) 或 上界(length-1)，返回 0（不可移动）
    # 否则返回 1（可移动）
    if(flag_now == lower or flag_now == file_play_len-1):
        return 0;
    else:
        return 1;
```

text_replace 区域清除

用于在更新文字前清除旧的文字区域（用黑色填充），避免文字重叠。

```
def text_replace(screen, last_rect):
    pg.draw.rect(surface=screen, color='black', rect=last_rect)
```

绘制函数

text_draw 绘制歌曲标题

负责在屏幕正中央显示当前选中的歌曲名称。

```
def text_draw(title_flag, font, screen_size, size_select):
    # 根据屏幕尺寸动态调整字号
    scale_factor = _get_scale_factor(screen_size)
    font_size = int(50 * scale_factor)
    font = pg.font.SysFont(None, font_size)

    # 获取当前标题
    text = title_song[title_flag]
    text_image = font.render(text, True, 'white')
    text_rect = text_image.get_rect()

    # 居中定位
    width, height = screen_size if isinstance(screen_size[0], int) else
screen_size[size_select]
    text_rect.center = (width//2, height//2)

    # 绘制
    surface = pg.display.get_surface()
    if surface is not None:
        surface.blit(text_image, text_rect)
    return text_rect # 返回矩形以便后续清除
```

attention_draw 绘制闪烁提示

绘制 "Press 'Enter' to start" 提示语，并利用时间戳实现闪烁效果。

```
def attention_draw(screen, screen_size, size_select):
    # ... (字号与位置计算) ...

    # 闪烁逻辑
    blink_time = pg.time.get_ticks() / 1000
    blink_speed = 1.5 # 周期 1.5秒
    blink_duration = 0.8 # 显示 0.8秒

    if(blink_time % blink_speed < blink_duration):
        screen.blit(attention_image, attention_rect) # 显示
    else:
        text_replace(screen, attention_rect) # 隐藏（清除）
```

独立运行测试

`if __name__ == "__main__":` 块提供了独立的选曲界面预览功能：

1. **初始化**：设置窗口，加载字体。
2. **首次绘制**：调用 `text_draw` 显示第一首歌。
3. **主循环**：
 - **边界检测**：调用 `flag_judge` 判断能否向上或向下翻页。
 - **事件处理**：
 - `K_DOWN`：若未到底，清除旧标题，`title_flag + 1`，绘制新标题。
 - `K_UP`：若未到顶，清除旧标题，`title_flag - 1`，绘制新标题。
 - `VIDEORESIZE`：响应窗口缩放，重绘界面。
 - **特效更新**：每帧调用 `attention_draw` 刷新提示语的闪烁状态。

play_interface_version2_final_version.py 游戏主逻辑

此文件包含游戏的核心玩法循环，负责音符的生成、下落、判定、得分计算、长条音符处理以及界面渲染。它是整个项目中最复杂、代码量最大的模块。

核心数据结构

全局状态变量

为了在不同函数间共享游戏状态，使用了大量全局变量（注：在大型项目中通常建议封装为类，但此处为了教学直观使用了全局变量）：

- `score` / `combo` / `max_combo`：记分系统。
- `rank_level_judge`：统计 Perfect/Good/Bad/Miss 的数量。
- `note_read_sp`：记录每个轨道（4轨）当前读取到了第几个音符，优化遍历性能。
- `column_statement` / `column_lock_clock`：用于长条音符（Long Note）的按压状态锁定。

初始化与预处理

note_time_initialize / note_rect_initialize

这两个函数在游戏开始前将 JSON 谱面数据转换为游戏可用的对象。

- `note_time_initialize`：解析每个音符的 `beat`（节拍），结合 BPM 计算出它们的**绝对出现时间**（秒），并按轨道分类存储到 `note_storage` 二维列表中。
- `note_rect_initialize`：根据音符类型创建 `pygame.Rect` 对象。
 - **普通音符**：创建高度为 10 的矩形。
 - **长条音符**：计算结束拍与开始拍的时间差，生成对应长度的长矩形。

first_note_time_calculate

计算第一颗音符到达判定线的时间，用于确定音乐播放的起始延迟，确保音画同步。

核心游戏循环 (run_game)

`run_game` 是外部调用的入口函数。它初始化 Pygame 窗口、加载音乐和谱面，然后进入 `while isRunning` 主循环。

1. 时间管理与同步

```
current_time = pg.time.get_ticks()/1000.0 - start_time + time_offset_sec
```

游戏的核心驱动力是 `current_time`。所有音符的位置、动画和音乐播放都依赖于这个经过校准的时间戳。

- `start_time`: 游戏开始时的系统时间。
- `time_offset_sec`: 包含用户校准的延迟 (Latency) 和谱面自身的偏移 (Offset)。

2. 音符逻辑 (note_judge & note_draw)

这是每一帧最繁重的任务：

1. **判定可见性 (note_judge)**: 检查 `note_storage` 中哪些音符的时间已经到了“进入屏幕”的时刻。将它们从“存储区”移动到“当前显示区” (`rect_note_current`)。
2. **位置更新 (note_draw)**:
 - 根据 `current_time` 和 `note_time` 的差值，乘以 `fall_speed`，计算每个音符当前的 Y 坐标。
 - **长条特殊处理**: 如果是一个正在被按住的长条，调用 `long_note_height_change`，根据按压时长实时缩短矩形的高度（模拟“吃掉”长条的效果）。
 - **Miss 判定**: 如果音符的 Y 坐标超过了屏幕下方（且未被击打），判定为 Miss，重置 Combo。

3. 输入判定 (note_keyboard_judge)

处理玩家的键盘事件 (`KEYDOWN` / `KEYUP`)。

- **普通音符**:
 - 当按下键时，检查该轨道最下方的音符。
 - 计算 `abs(note_time - current_time)`。
 - 调用 `rank_judge` 根据时间差判定 Perfect (<50ms), Good (<80ms), Bad (<120ms) 或 Miss。
- **长条音符**:
 - **按下 (KEYDOWN)**: 判定头部的时间差，如果命中，设置 `column_lock_clock` 记录开始按压时间，并将 `note_duration_time` 设为长条时长。

- **抬起 (KEYUP)**: 检查是否过早松开。如果长条还没结束就松手，会触发 Combo 中断（视为断连）。

4. 视觉反馈

- **判定线**: 绘制固定的 4 条轨道线和底部黄色的判定线。
- **文字特效 (text_draw)**: 在屏幕中央显示当前的评价 (Perfect/Good...) 和 Combo 数。包含淡出 (Alpha) 动画逻辑。
- **UI 面板 (draw_score_display)**: 在左上角实时更新分数、准确率和各判定数量。
- **进度条 (draw_progress_bar)**: 在底部绘制歌曲进度，当进度条满时触发游戏结束逻辑。

5. 自动播放 (Auto Play)

代码中包含了一个隐藏的 Auto Play 功能（按 Q 键切换）。

```
if auto_play_enabled:
    # 遍历当前音符，如果时间差 < 0.02秒，自动调用 note_keyboard_judge 模拟按键
    if abs(note_current[lane][0] - current_time) <= 0.02:
        # ... 模拟按下 ...
```

这对于测试谱面和判定逻辑非常有用。

6. 暂停与恢复

当按下 ESC 时，游戏进入暂停状态。

- 记录 freeze_elapsed，在恢复时修正 start_time，确保暂停期间游戏时间不流逝。
- 从暂停返回时，激活 resume_metronome_active，播放 4-8 拍的节拍器倒计时，给玩家反应时间，然后再恢复音乐和音符下落。

结果结算

当音乐播放完毕且所有音符处理完成后，函数打包当前的统计数据 (Score, Accuracy, Perfect数等)，返回给 main.py 以便显示结算画面。

```
return {
    'title': title_song,
    'score': score,
    'accuracy': accuracy,
    # ...
}, local_offset
```

result.py 结算界面

result.py 负责在单局游戏结束后展示详细的成绩统计，包括分数、评级、Max Combo 以及具体的 Perfect/Good/Bad/Miss 数量。

引入库与辅助函数

```
import pygame as pg
import sys
```

`_grade_from_accuracy` 评级计算

根据准确率 (Accuracy) 返回对应的等级字符串。

- **SS**: 100%
- **S**: $\geq 98\%$
- **A**: $\geq 95\%$
- **B**: $\geq 90\%$
- **C**: $\geq 80\%$
- **D**: $< 80\%$

`_grade_color` 评级颜色

为每个等级返回对应的 RGB 颜色元组，例如 SS 为金色 (255, 215, 0)，D 为红色 (255, 120, 120)。

`_draw_vignette` 晕影效果

为了让界面看起来更有质感，这个函数在屏幕上绘制一个径向渐变的黑色遮罩 (Vignette)。

```
def _draw_vignette(screen, strength=110):
    # ... 创建带 alpha 通道的 surface ...
    # 从中心向外绘制透明度逐渐增加的圆环
    for r in range(max_radius, 0, -40):
        # ...
        pg.draw.circle(overlay, (0, 0, 0, alpha), center, r)
```

主逻辑函数 (run_result)

接收 `result_data` (由 `play_interface` 生成的字典) 并渲染界面。

参数:

- `result_data`: 包含 score, max_combo, accuracy 等数据的字典。
- `screen_size`: 当前窗口大小。

绘制布局:

界面布局采用了相对坐标与固定偏移结合的方式，利用 `_get_scale_factor` 确保在不同分辨率下元素位置合理。

- **Top**: 显示巨大的 "RESULT" 标题。
- **Center**:

- 中央显示硕大的等级字母（如 S）。
- 字母下方显示总分（Score）。
- 左侧显示 Max Combo 和 Accuracy。
- 右侧分列显示详细判定数（Perfect, Great, Bad, Miss）。
- **Bottom:** 显示歌曲名称和 "Retry / Back" 选项按钮。

交互逻辑:

- ↑/↓: 切换 Retry 或 Back 选项。
- **Enter**: 确认选择。返回字符串 "retry" 或 "back" 给主程序。
- **Esc**: 作为 Back 的快捷键。

此界面的设计重点在于清晰的信息层级，让玩家一眼能看到最重要的等级和分数，同时也能查阅详细的发挥情况。

auto_play_bot.py 自动游玩脚本 (外挂)

这是一个独立于游戏主程序的外部 Python 脚本。它不通过读取内存或游戏代码运行，而是像人类玩家一样“看”屏幕并“按”键盘。这通常被称为“视觉脚本”或“物理外挂”。

引入库

```
import mss          # 用于极速屏幕截图
import numpy as np  # 用于高效处理图像数组
import keyboard     # 用于模拟键盘按键 (A/S/K/L)
```

核心配置

脚本开头定义了针对特定分辨率（2560x1600）的坐标参数。如果要适配你的屏幕，需要修改这些值。

```
Y_SCAN = 1373      # 判定线的 Y 轴坐标（扫描这一行像素）
X_LANES = [1031, 1190, 1324, 1470] # 四个轨道的 X 轴坐标
BLUE_THRESHOLD = 100 # 判定阈值（蓝色通道值）
```

工作原理 (Main Loop)

脚本运行在一个无限循环中，追求极致的响应速度。

1. 屏幕捕获 (mss)

```
with mss.mss() as sct:
    monitor = {"top": Y_SCAN, "left": 0, "width": 2560, "height": 1, ...}
    img = np.array(sct.grab(monitor))
```

- 它只截取判定线所在的这一行像素（Height = 1）。

- 相比截取全屏，这种方式数据量极小，处理速度非常快，能实现极低的延迟。

2. 视觉分析

遍历四个轨道对应的 X 坐标，检查该点的像素颜色。

```
blue_value = img[0, x, 0] # 获取 BGR 中的 Blue 分量
note_detected = blue_value > BLUE_THRESHOLD
```

- 游戏中的音符是白色的 (RGB 255,255,255) 。
- 背景是深色的。
- 只要蓝色分量足够高 (>100) ，脚本就认为“音符到了”。

3. 模拟输入

```
if note_detected:
    if not key_states[i]:
        keyboard.press(KEYS[i]) # 模拟按下
        key_states[i] = True
else:
    if key_states[i]:
        keyboard.release(KEYS[i]) # 模拟松开
        key_states[i] = False
```

- **按下逻辑**：当检测到音符且当前按键未按下时，触发 `press`。
- **松开逻辑**：当音符消失（像素变暗）且当前按键是按下状态时，触发 `release`。
- 这种逻辑天然支持**长条音符**（Long Note），因为长条经过判定线时像素一直保持亮色，脚本就会一直按住不放。

使用说明

此脚本需要独立运行，并且通常需要管理员权限（在 Windows 下）才能向全屏游戏发送按键指令。